CSCI160 Computer Science I
Lab Exercise: Triangle and Season Classification

Objectives

- Understand and evaluate nested boolean expressions
- Implement conditional logic using `if` statements, including nested conditions
- Debug and test code with test cases
- Practice good coding style with appropriate white space and comments

1. Evaluate Simple Boolean Expressions

Determine the result for each of the following boolean expressions, given values for the variables. Check your conclusions by running these as in the Python interactive shell.

Submit your answers either as a text document or as comments in your code for the later problems.

Values: `A = 5`, `B = 12`, `C = 13`

1. `(A + B) > C`
2. `(A + C) > B`
3. `(B + C) > A`
4. `(A == B) AND (B == C)`

Values: `month = 3, day = 20`

1. `month >= 3 and month <= 6`
2. `day >= 20 and day <= 31`
3. `month == 12 and day >= 21`
4. `month < 1 or month > 12`

2. Evaluate Nested Boolean Expressions

Given the following boolean expressions, determine the result for the given values of the variables

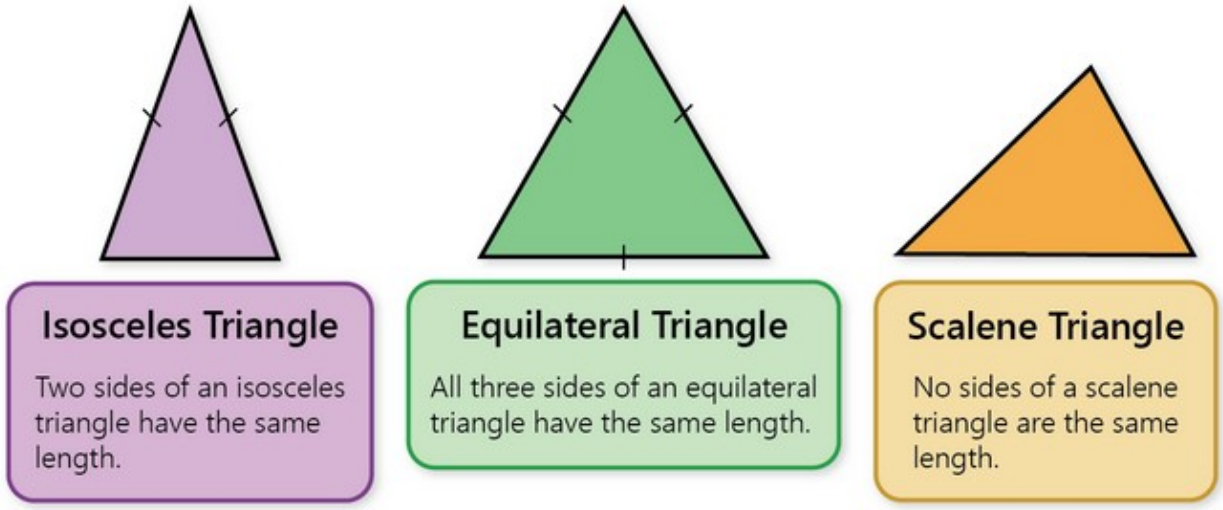
Values: `A = 7`, `B = 24`, `C = 25`

1. `(A + B > C) AND (A + C > B) AND (B + C > A)`
2. `(A == B) OR (B == C) OR (A == C)`
3. `(A**2 + B**2 == C**2) OR (A**2 + C**2 == B**2) OR (B**2 + C**2 == A**2)`

Values: `month = 9, day = 22`

1. `(month == 9 and day >= 22) or (month > 9 and month < 12)`
2. `(month == 12 and day >= 21) or (month < 3)`
3. `(month == 6 and day >= 21) or (month > 6 and month < 9)`

PROGRAM: Triangle Classification – Problem Statement



AI Statement: To get the most out of this assignment you should attempt this yourself, without using AI. If you do use AI you are required to cite its usage as described in the course outline. Failure to do so will be considered a violation of academic integrity.

You are also not allowed to submit someone elses code as if it were your own. People have been doing this and this is considered cheating.

Objective:

Write a Python program that takes three triangle side lengths as input and determines whether they form a triangle. If they do, classify the triangle as **equilateral**, **isosceles**, or **scalene**. Additionally, check if the sides form a **Pythagorean triple**.

Requirement: You are required to stick with the techniques we are currently studying. Do not submit advanced coding techniques that use Python modules that simply solve the task for you. This will be penalized. If you are unsure, please ask.

1. Problem Breakdown:

1. **Starter Code**
    - Download the Python file named: **triangle.py**.
    - This file contains a tiny bit of starter code and some test cases and general info on input validation.
    - Add your header to the file and include a brief description of the problem.
    - **Write Python code to implement the solution as defined below.**

2. **Input Collection**
    - Prompt the user to enter three side lengths: `a, b and c`
3. **Triangle Validity Check**
    - Use the following conditions to determine if the sides form a valid triangle. Any of these conditions can be true
        - `a + b > c`
        - `a + c > b`
        - `b + c > a`
    - If the sides do **not** form a triangle, print: `{a}, {b}, and {c} do not form a triangle` and do not allow any further code to execute.
4. **Triangle Classification**
    - If the sides form a triangle, classify it as:
        - **`Equilateral`**`: All sides are equal.`
        - **`Isosceles`**`: Exactly two sides are equal.`
        - **`Scalene`**`: No sides are equal.`
    - Print the appropriate message:
        - `{a}, {b}, and {c} form an equilateral triangle`
        - `{a}, {b}, and {c} form an isosceles triangle`
        - `{a}, {b}, and {c} form a scalene triangle`
5. **Pythagorean Triple Check**
    - If the sides form a triangle, check if they also form a **Pythagorean triple**.
    - A Pythagorean triple satisfies the condition: $a^2 + b^2 = c^2$ where `c` is the longest side.
    - Determine the longest side dynamically using conditional logic. You cannot prompt for the longest side specifically. It can be either a, b or c at any given run.
    - Print the appropriate message:
        - {a}, {b}, and {c} form a Pythagorean triple
        - {a}, {b}, and {c} do not form a Pythagorean triple

2. Handling User Input

- Ensure the user enters ***valid positive numbers*** for the side lengths.
- If the input is invalid (e.g., negative numbers or non-numeric input), display an error message and do not allow any further code to execute. The program should terminate. There are some hints to doing this in starter code.

4. Test Cases

- The starter code has a collection of test cases for Pythagorean triples. Your code should correctly identify all of these.

5. Sample Runs

Input **Output**

```
97          97, 65, and 72 form a scalene triangle
65          97, 65, and 72 form a Pythagorean triple
72
```

Input **Output**

```
200         200, 1, and 17 do not form a triangle
1
17
```

Input **Output**

```
3           3, 3, and 4 form and isosceles triangle
3           3, 3, and 4 do not form a Pythagorean triple
4
```

PROGRAM: Date Validation and Season Determination – Problem Statement

Objective:

Write a program that takes a date as input in `MM/DD/YYYY` format and outputs the corresponding season. Additionally, validate the month and day to ensure they are within valid ranges.

1. Problem Breakdown:

1. Download the Python file named: **seasons.py**.
    - This file contains starter code, including a list of months and placeholders for the date validation and season determination logic.
    - Add your header comment block to the top of the file. Include a brief description of the problem.
    - **Write Python code to implement the solution as defined.**
2. **Input Collection**
    - Prompt the user to enter a date in `MM/DD/YYYY` format.
3. **Date Validation**
    - Check if the month is valid (1–12). If not, print: `Invalid Month` and exit the program

- Check if the day is valid for the given month. If not, print: `{Month}: Invalid Day` and exit the program

4. **Season Determination**
    - Use the following rules to determine the season:
        - **`Spring`**: `March 20 – June 20`
        - **`Summer`**: `June 21 – September 21`
        - **`Autumn`**: `September 22 – December 20`
        - **`Winter`**: `December 21 – March 19`
    - Print the season in the format: `{Month}: {Season}`
    - **Use list membership for `"or logic"`:** If you place months into lists you can use the membership operator `(in)` to easily check for true.
        - **List membership:** `if month in ["January", "February"]`
        - **Explicit or logic:** `if month == "January" or month == "February"`

3. Test Cases

Test your program with the following inputs:

Input Date	Expected Output
04/11/2020	April: Spring
06/21/2020	June: Summer
09/22/2020	September: Autumn
12/21/2020	December: Winter
02/29/2020	February: Invalid Day
13/01/2020	Invalid Month
08/37/2020	August: Invalid Day
03/20/2020	March: Spring
06/20/2020	June: Spring
06/21/2020	June: Summer
09/21/2020	September: Summer
09/22/2020	September: Autumn
12/20/2020	December: Autumn
12/21/2020	December: Winter
03/19/2020	March: Winter

Lab Deliverables - Check the rubric to ensure you have everything

1. A Python file named **seasons.py** with your implementation.
2. A Python file named **triangle.py** with your implementation.
3. A document with answers to the boolean expression questions (Sections 1–2).